

Introduction

Background Problem

Crew is an airline's second-largest operating cost, behind only fuel, so how flights are staffed has a real effect on profitability. The cabin-crew pairing problem asks for a set of legal duty sequences, or *pairings*, that each begin and end at a crew member's home base, respect working-time rules, and together cover every scheduled flight at minimum cost. Cabin crew are harder to schedule than cockpit crew because they are heterogeneous. They are cross-qualified across aircraft types and split into classes, and how many of each class a flight needs depends on the aircraft's size and cabin layout. A regional turboprop might need a single attendant, while a widebody needs eight.

We model this in a deliberately simple form. Each flight needs between one and eight crew. Exactly one must be a *senior*, the rest can be anyone. A senior can fill any seat, but a normal can never fill the senior's. The senior is therefore the scarce, binding resource: it decides whether a flight can be staffed at all.

A pairing is legal only if it respects the operational clocks the model tracks. Two consecutive flights need at least a 45-minute turnaround between them. A duty period can build up at most 14 hours of block time before an 8-hour rest clears it. And a crew member can stay away from base for at most 4 days before owing a 48-hour home break.

Unlike formulations that plan around a single base and import "extra" crew at a flat penalty, we model the real geography: crew are based at many airports, can only originate and terminate duties at their own base, and must be physically positioned, by flying or deadheading, to wherever a flight departs. Coverage is thus limited not just by how many crew exist, but by whether the right class can legally reach the right place at the right time. That is the central tension the rest of this report addresses.

Original Paper's methodology

Problem Setups

Flight Data Source

The flight schedule is real U.S. domestic data from the Bureau of Transportation Statistics (BTS) on-time performance dataset, which lists for every flight f its operating carrier, tail number, origin and destination, scheduled departure and arrival times, and distance. What BTS does not record is how many cabin crew a flight needs, so we derive it from the aircraft itself. Each tail number is joined against the FAA Aircraft Registry to recover the aircraft model and its seat count s_f – the per-tail registered value where available, otherwise a type-level estimate from FAA Type Certificate Data Sheets – and s_f is mapped to a minimum cabin crew r_f under U.S. regulation 14 CFR 121.391, where one attendant is required for every 50 seats:

$$r_f = \lceil s_f/50 \rceil$$

After enrichment the dataset spans 21 carriers, from mainline operators down to small regionals. This gives a natural range of network sizes and crew requirements to test against. We treat each carrier independently, with its own flights, airports, and crew pool, so an airline is one self-contained instance. The planning period is 30 days, extended by a return tail to a 34-day horizon so crew committed late can still be routed home. Only flights departing within the planning period must be covered; the rest sit in the tail.

The crew requirement is uneven across carriers, but within a single regional carrier it is essentially constant. A regional flies a near single-class fleet: one aircraft type, one seat band. So every flight

maps to the same r_f . G7 is two attendants throughout (its regional jets seat 51 to 100), as are ZW and QX. Real per-flight variation appears only at mainline scale, where the fleet spans seat classes. B6 ranges over two to four, HA three to six, and AA three to eight. Size and density climb the same way. The regionals are small and sparse: G7 touches 51 airports over 143 routes, ZW 44 over 93. A mainline such as AA spans 119 airports and 836 routes at several times the density (Table 1).

Carrier	Flights	Airports	Routes	Flights / airport	Min. crew r_f
ZW	3,790	44	93	86	2 (uniform)
G7	5,575	51	143	109	2 (uniform)
C5	6,612	57	116	116	2 (uniform)
YV	6,628	69	178	96	2 (uniform)
HA	6,690	22	80	304	3–6
QX	7,754	53	201	146	2 (uniform)
G4	9,345	119	789	79	3–4
PT	10,621	69	175	154	2 (uniform)
F9	15,526	80	624	194	3–4
NK	17,544	60	433	292	3–4
B6	17,918	57	281	314	2–4
AS	18,163	85	357	214	3–4
9E	18,279	107	380	171	2 (uniform)
OH	21,092	94	375	224	2 (uniform)
MQ	21,888	145	526	151	2 (uniform)
YX	27,854	83	600	336	2 (uniform)
UA	62,007	119	811	521	3–8
OO	65,026	239	1,231	272	2 (uniform)
AA	75,088	119	836	631	3–8
DL	76,306	140	879	545	2–7
WN	105,307	104	1,606	1013	3–8

Table 1: Network size, density (flights per airport) and per-flight crew requirement for every operating carrier in the January 2025 BTS data. *Routes* counts distinct directed origin–destination pairs.

This leaves an awkward trade-off when picking test instances. The carriers with genuinely varied requirements (B6, HA, AA) are exactly the ones too large to solve in reasonable time. The small regionals that do solve have a flat requirement that never stretches the senior/normal split past a fixed one-plus-one. So we use two complementary instances. On the real data we focus on G7. It is a regional small enough to solve, and its true requirement of two (one senior, one normal) still puts the full two-layer machinery to work on a genuine schedule. For varied demand we keep that same small ZW network but swap its requirement for a random r_f , uniform over 1 to 8 (data/ flights_2025-01-random.csv). That gives the mixed demand of a mainline on a network small enough to solve. It exercises cancellation, multi-seat fill and substitution in ways the uniform regionals never would.

Two Layer structure

Each flight needs one senior plus $r_f - 1$ normals. Rather than solve for both at once, we run two passes that each place a single crew member per flight. This is about tractability. The crew-flow relaxation is integral only at unit demand, so the barrier method reaches an integer optimum without branching. The moment a flight needs two or more, the coverage constraints tie the flow across crew groups together, the relaxation turns fractional, and the solver has to branch. Treating the seniors (one per flight) and the normals ($r_f - 1$ per flight) as separate unit-demand passes keeps both in that easy regime. We solve the seniors first because they are the binding resource; the normals then fill in around the schedule the seniors fix.

Senior Layer (1 senior per flight)

Pass one stamps every flight with demand one and solves over a dedicated senior pool. Each flight gets exactly one senior, or none if none can legally reach it. A flight that gets no senior is *cancelled*: a normal can never fill the senior seat, so it is dropped and reported uncovered. The rest pass to the normal layer.

Normal Fill Layer ($r_f - 1$)

Pass two takes those surviving flights, re-stamps each with demand $r_f - 1$, and solves over a separate normal pool. Flights with $r_f = 1$ are already complete and carry no demand here; $r_f = 2$ stays unit-demand, higher values give small multiplicity. Normal identifiers are offset so they never collide with senior ones on merge.

Senior Substitution in Idle Gaps

After both passes, some surviving flights are still short a normal. A senior can fill a normal seat, but only opportunistically: inside an idle gap of its own layer-1 route, never displacing a senior duty. This is a greedy post-processing pass, not part of either MIP. It fills one seat per gap and writes each accepted fill back as a real flight leg on the senior's route, so the schedule, visualiser and validator all reflect it.

An idle gap runs from a senior's arrival at an airport up to its next senior departure, or to the end of its route. A candidate fill flight f must clear two gates.

Geometric Feasibility

The senior must be parked at f 's origin, rested across the gap (the 8-hour minimum, so $\text{dep}_f - \text{arr}_{\text{prev}} \geq 480$ min), and, when a senior duty follows the gap, able to connect to it: f must land where that duty departs, leaving the 45-minute turnaround, $\text{arr}_f + 45 \leq \text{dep}_{\text{next}}$. A gap with no following duty clears this trivially.

Route Legality

This is the gate geometry alone misses: a senior parked far from base could satisfy the geometry yet break the away cap by flying a fill hours or days later. So we insert f into the senior's route, re-sort by departure, and re-check the whole route against the same rules the model enforces. No leg may depart more than 4 days after the senior last left home with no 48-hour break in between, and no run of consecutive duty days may exceed the limit. The check is cumulative, so several fills on one senior stay jointly legal. A gap failing either gate yields nothing, which makes the pass best-effort: it recovers a handful of understaffed flights and leaves the rest short.

Crew Base Allocation

Choosing where crew are based and how many to base there is one heuristic, run once per layer. The base set is every airport the carrier flies both to and from; each such airport is a full crew base.

For a base p we estimate two loads. The first is its duration-weighted demand, the crew-minutes that originate there:

$$D_p = \sum_{f: \text{orig}(f)=p} m_f d_f$$

where m_f is the layer's per-flight demand (one for the senior pass, $r_f - 1$ for the normal pass) and d_f the block duration. The second is the peak concurrent load P_p , the largest number of crew on duty at p at once, found by sweeping the day and adding m_f at each departure, removing it at each arrival.

The pool must exceed the raw demand, since crew are not available the whole horizon. An 8-hour overnight rest separates duty days, and a 48-hour home break follows every rotation of at most 4 days away, so over a six-day cycle only about four days are workable. A utilisation factor $u = 0.55$ absorbs these overheads; it is hand-tuned rather than derived, set empirically so busy bases are not undersized. For sizing we credit each crew a nominal eight duty-hours a day, well below the enforced 14-hour cap, so over the H -day horizon one crew supplies $\tau = 8\text{h} \times H \times u$ duty-minutes. This eight-hour figure is a sizing assumption only; the schedules themselves are bound by the 14-hour duty limit. The base size is then

$$n_p = \max(\lceil 1.8 D_p / \tau \rceil, \lceil 1.8 P_p \rceil, n_{\min})$$

a 1.8 slack over the larger load, floored at a small per-base minimum $n_{\min}$, with a 10% Gaussian jitter on each count. The 1.8 covers positioning (a crew often deadheads to where it is needed) and short demand peaks. It is ample: the pools supply two to nearly five times the crew-minutes flown (a coverage ratio of 2.8 for G7, around four for the randomised ZW), so headcount is never binding. Coverage is limited by geography and legality, not numbers, so the slack could even be tightened to shrink the model.

Running this twice, on the senior demand and then the normal demand, gives two pools each matched to its own layer. Sizing one pool and splitting it by a fixed ratio was tried first, and it mismatched both layers.

Formulations

Set

Data

Rolling Horizon

Seaming

Window Carry-over

At each seam the next window starts from the committed state of the previous one. Two things carry across: each crew's committed end position – the airport it occupies when the committed region closes – and its break-clock state. The clock is summarised by a single anchor, the minute the crew last left home after a completed home break, from which the away budget keeps counting. The subtlety is what counts as a completed break: only a 48-hour home stay whose full 48 hours elapse inside the committed region advances the anchor. A break the solver schedules in the uncommitted tail is deliberately not credited, because the tail is re-planned by the next window and that break may never actually be flown. A crew that is home at the seam but has served only part of its break carries the start of that home stay forward, so the next window finishes the remaining hours rather than restarting a fresh 48 – this neither forces an early return nor grants a free reset.

Crew-Flow Model

Break-Clock Expansion

Clock-Group Aggregation

Integer Group Flow

Recovering Schedules by Flow Decomposition

Acyclic Graph gives a Clean Path Split

Coverage is Preserved

Break Requirements are Preserved

Connectivity from Flow Conservation

Interchangeability and Arbitrary Assignment

Object Function

Uncovered-Slot Penalty

Constraints

Coverage

Turnaround (45 min)

Duty Limit (14h block time since last rest)

Overnight Rest (8h)

Home Break (48h) and Away Cap (4 days)

Solve Method

Barrier vs Simplex

The per-window relaxation is large and sparse, so it is solved by the barrier (interior-point) method rather than simplex, which otherwise spent the whole time limit pivoting at the root. For the unit-demand models – the senior layer, or any single-crew airline – the group-flow relaxation is integral, so the barrier lands directly on an integer optimum, branch-and-bound never fires, and crossover to a simplex basis is pure overhead and is switched off. This is the regime the method is built for, and such windows solve in well under a minute. Once a flight needs more than one crew the coverage constraints couple flow across groups and break that integrality, so the root relaxation is fractional and the model must branch, with the consequences taken up in the results.

Deterministic Model Construction

The expanded graph is built by exploring states held in hash sets, whose iteration order depends on the process hash seed. Identical inputs therefore produced models with their variables in different column orders from one run to the next, and because the crew-flow model is highly degenerate – many interchangeable crew and equivalent paths – the solver's anti-degeneracy effort swung sharply with that order, the same window taking anywhere from thirty seconds to over a hundred. Sorting arcs and nodes by value before they are handed to the solver makes the constructed model byte-identical across runs – same fingerprint, same result, same time – which removes the variance,

makes timings comparable, and as a side effect presolves slightly smaller, since the regular ordering is easier to reduce.

Results

G7

Coverage Breakdown

Fully Crewed

Cancelled (No Senior)

Understaffed (Normal Shortfall)

Uncovered flights

Structural Spokes (isolated, sub-45 turnaround)

One-Directional Spoke Connectivity

Rolling-Horizon Seam Effects

Senior as the Binding Resource

Senior Substitution Outcomes

Two-Layer Tractability

Multi-Crew as Two Unit-Demand Layers

Effect of Barrier on Large Windows

Solving the multi-crew requirement directly – one model with $r_f > 1$ – shows why the two-layer split matters. With every G7 flight needing two crew, the coverage constraints make the root relaxation fractional, so the barrier point is no longer integer and the model must branch. Most windows still resolve at the root, but the windows late in the horizon, where the return tail runs past the end of the flight data and connectivity is sparse, stall: the relaxation bound is loose – the LP “covers” flights with fractional flow that no whole crew can realise – and, with crossover off, each branch-and-bound node re-solves its LP without a warm-start basis, so node throughput collapses. Three of the ten windows hit the thirty-minute limit at 24 to 83 per cent gap with badly degraded coverage. The two-layer decomposition avoids this entirely: each layer is unit-demand and therefore root-integral, so every window solves quickly – the same airline that stalls as one model runs cleanly as two layers.

Schedule Validation

Rule-Compliance Checks

Window-Boundary Carry-over Fix

An earlier carry-over read each crew’s break deadline straight off the last committed arc of the expanded graph. Because the cost-minimising solution within a window is free to park the mandatory 48-hour break in the uncommitted tail, that arc reported a deadline as if the break had already been served, so the anchor advanced every window even though no break was ever committed. The away clock therefore reset at each seam and drifted forward, letting crew accumulate five to seven days away and double-digit consecutive duty days across the merged schedule – on the G7 instance the independent validator flagged 352 away-cap and 341 consecutive-

duty violations, none of which were visible window-by-window. Tracking the anchor on the committed legs only, exactly as the validator reconstructs it, and crediting a partly-served break across the seam, removes the drift: the same instance then validates with zero violations of any rule while senior coverage is essentially unchanged at 5001 of 5141 flights, confirming the violations were a boundary-accounting artefact rather than illegal routes the solver actually wanted.

Model Scaling Limits

Things Attempted but Left Out

Per-crew Binary

Direct Multi-Crew ($\text{min_crew} > 1$) Solve

Barrier with Crossover Disabled

Enabling crossover globally, to give branch-and-bound a warm-start basis on the fractional multi-crew windows, was a net loss. The easy windows – which solve at the root and never branch – paid a large crossover cost for nothing: forcing a basis on a million-variable degenerate model meant pushing hundreds of thousands of variables to a vertex (with a restart), and one window grew from about seven to twenty-two minutes for no benefit. A targeted variant was kept as an opt-in instead: probe with crossover off, and only if a window times out while still branching far from optimal, switch crossover on and re-solve with the full budget. This rescues the mid-size stalled windows without taxing the majority, but it does not help the largest window, which is stuck in root processing before branching even starts – a model-size problem, not a missing-basis one.

Looser Time Limit

Mainline-Scale Airlines

DDD dead loop

A Separate counter for working days for each time away window

- It's basically the same thing as timeaway but blows up the model quite a bit
- merged into mandatory 48h home break every 4 days

Longer Away Window (7-day $\rightarrow$ 4-day)

- the return tail T_{tail} equals D_{AWAY} , so a 7-day away cap forces a 7-day tail $\rightarrow$ 14-day window horizon (vs 11-day at 4 days) $\rightarrow$ much larger graph/model per window
- dropped to a 4-day away cap (mandatory 48h home break), shrinking each window
- also let the separate consecutive-duty-days counter fold in (above)

Cutting Planes

vry slow, and it kinda didnt work.... got clean violations

RULE-VIOLATION SUMMARY

continuity : 0 violation(s) overlap : 0 violation(s) turnaround : 0 violation(s) d_away : 0 violation(s)
home_break : 0 violation(s)

TOTAL: 0 violations

but its cuz the d_away cut holding everyone under 4 days, the “48h break after a 4-day rotation” rule will essentially never fire — crew instead rest at home in shorter, more frequent stretches 301ed51

Warm Starts

Warning: Completing partial solution with 773441 unfixed non-continuous variables out of 773446

hint prior-window routes via VarHintVal instead of partial Var.Start, dropping Gurobi's 80s completion sub-MIP

diff warm start attempts through three stages. F

the Gurobi log showed the original `_apply_warm_start` was setting `Var.Start=1` on only matched flight arcs and leaving every wait arc undefined, producing a near-empty partial start — Completing partial solution with 773441 unfixed ... out of 773446 (only 5 variables fixed). Gurobi reacted by running a completion sub-MIP that burned the whole time budget (0 nodes explored in subMIP for 300s, first incumbent at 247s). So I rewrote it to reconstruct each crew's complete in-window flow by walking the wait-arc spine (depot → wait → leg → wait → leg ...) and added instrumentation reporting match rate and coverage.

That instrumentation produced the key evidence: matching was **perfect** (legs 11/11 matched, 14/14 matched) but only 11–14 legs carried across each window boundary, hinting just 0.1–0.4% of variables (2320/619830, 939/788706). The cause is structural, not a bug — each window only flies its committed 3-day region and leaves the 4-day overlap idle because tail flights aren't in `F_cov` and so carry no coverage reward, leaving almost nothing to seed. Worse, even that sparse start still triggered the completion sub-MIP (Completing partial solution with 617795 unfixed, then 80s of 0 nodes yielding a throwaway $2.02e8$ incumbent), making the warm start net-negative: window 1 with it ran 129.6s versus window 0 without it at 106.1s. The fix was to switch from `Var.Start` to `VarHintVal` — hints feed the same guidance to Gurobi's heuristics without invoking the completion sub-MIP, removing the 80s overhead.

Future Work / Proposals

Two-Phase (Lexicographic) Objective

Joint Senior + Normal Two-Commodity Flow

Flow-Based Senior Substitution

Coordinated Two-Layer Positioning

Scaling to Mainline-Size Airlines

Shorter Window Horizon

Coarser Time Bucketing

Column Generation

Deterministic Tie-Break for Trajectory Stability

Realistic `min_crew` Data